KL divergence, Cross-Entropy & Augmentation

김영은

Latest Feedback

~~개념과 수식에 익숙해지기~~
~~한번 더 읽기 → 장기기억에 넣기~~

~~모든 dataset에 다 사용할 수 있다면, 단점은 뭐일지?~~

~~UMAP사용한 논문 읽어보고, 사용하지 않았을때와 성능 비교해보기~~

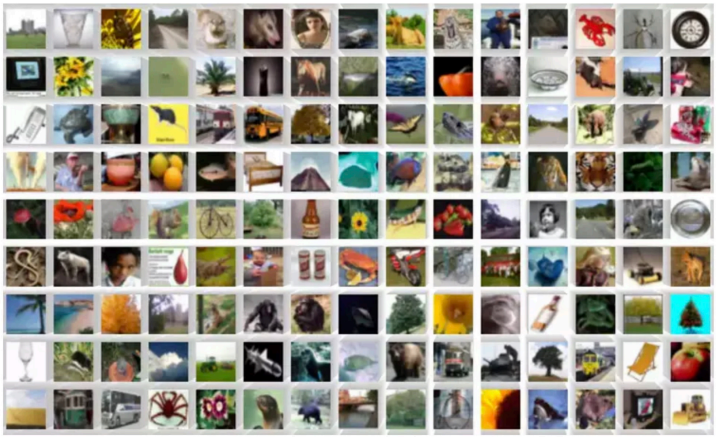
Dataset

CIFAR-100

데이터 수: 총 60,000개의 32x32 픽셀 크기 컬러 이미지.

클래스: 100개의 개별 클래스, 각 클래스는 500개 train, 100개의 test 로 구성

100개의 클래스는 20개의 상위 카테고리로 그룹화 → 계층적 구조



과정

1. 데이터 로드

2. 정규화 : 이미지의 각 픽셀 값(0~255)을 255로 나누어 0과 1 사이의 실수 값으로 만듦
   → 빠르고 안정적인 학습

3. 원-핫 인코딩 : 컴퓨터는 문자를 이해하지 못하기 때문에 숫자 레이블을 벡터 [0,0,0,0,1,0,. . .] 형태로 바꿈
   → 정답을 확률 분포로 계산하기에 용이

4. CNN 설계 & 학습

CNN

1. 데이터 로드

2. 정규화 : 이미지의 각 픽셀 값(0~255)을 255로 나누어 0과 1 사이의 실수 값으로 만듦
   → 빠르고 안정적인 학습

3. 원-핫 인코딩 : 컴퓨터는 문자를 이해하지 못하기 때문에 숫자 레이블을 벡터 [0,0,0,0,1,0,. . .] 형태로 바꿈
   → 정답을 확률 분포로 계산하기에 용이

4. CNN 설계 & 학습

Test 해석

성능 비교

- Cross-entropy accuracy : 34.82%
- KL divergence accuracy : 34.32%

→ 분류 문제에서 수학적으로 거의 유사한 역할

Cross-entropy & KL divergence accuracy

Cross-Entropy

실제 정답 분포(p)와 모델의 예측분포(q) 사이의 차이 측정

→ 모델이 얼마나 예측을 못했는지 나타내는 loss

$$H(p,q) = -\sum_{i=1}^{N} p(x_i)\log(q(x_i))$$

N : 전체 클래스의 개수 (CIFAR-100에서 100개)

$p(x_i)$: i번째 클래스가 실제 정답일 확률 (One-Hot 벡터에서는 정답 클래스는 1, 나머지는 0)

$q(x_i)$: 모델이 i번째 클래스라고 예측한 확률 (Softmax 출력값)

Cross-entropy

예시: 3개 클래스 (고양이, 개, 새) 분류

- 정답 (p): 고양이 → [1, 0, 0]
- 모델의 예측 (q): [0. 7, 0. 2, 0. 1] (고양이일 확률 70%)

이때 Cross-Entropy 손실 계산

→ $H(p,q) = -(1 \times \log(0.7) + 0 \times \log(0.2) + 0 \times \log(0.1))$

$H(p,q) = -\log(0.7) \approx 0.357$

KL divergence accuracy

두 확률분포가 얼마나 다른지, 정보 이론 관점에서 '거리'를 측정

$$D_{KL}(p||q) = \sum_{i=1}^{N} p(x_i) log(\frac{p(x_i)}{q(x_i)})$$

동일한 예시: 3개 클래스 (고양이, 개, 새) 분류

- 정답 (p): 고양이 → [1, 0, 0]
- 모델의 예측 (q): [0. 7, 0. 2, 0. 1] (고양이일 확률 70%)

KL divergence 손실 계산

→ 정답(p)에서 값이 0인 부분은 계산에서 제외, 첫번째 클래스 ('고양이') 만 계산하면 됨

KL divergence accuracy

$$D_{KL}(p \parallel q) = 1 \times log(\frac{1}{0.7})$$

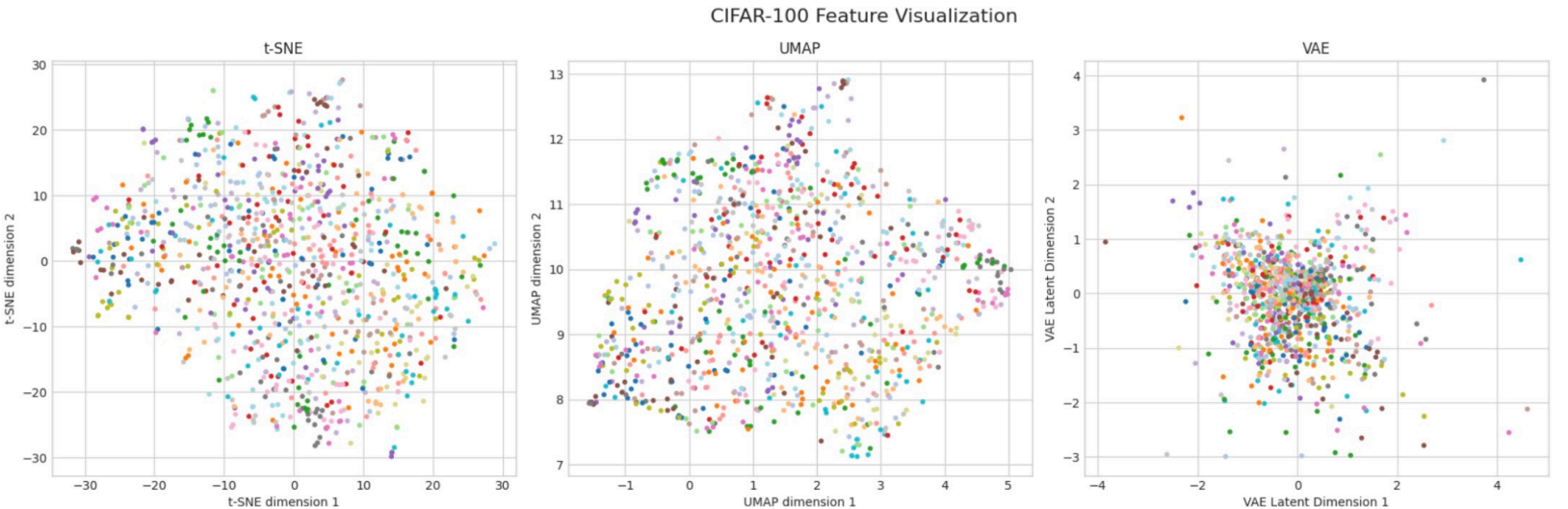
로그 속성 $\log(\frac{a}{b}) = \log(a) - \log(b)$ 이용하면,

$$D_{KL}(p \parallel q) = log(1) - log(0.7) = -log(0.7) \approx 0.357$$

→ Cross-Entropy 계산과 동일 (정답이 원핫일 때에는 동일)

t-SNE, UMAP and VAE

고차원 공간에서 점들 사이의 관계를 저차원 공간에서 점들의 관계로 잘 표현하는 것



t-SNE (t-Distributed Stochastic Neighbor Embedding)

고차원에서 가까웠던 이웃은 저차원에서도 반드시 가까운 이웃이어야 한다 (지역적 관계 보존)

KL divergence를 최소화함이 목표

$$L_{t-SNE} = D_{KL}(P||Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

p_{ij} : 고차원 공간에서 점 i 와 점 j가 이웃할 확률 (거리와 값 반비례, 정규분포 기반 계산)

q_{ij} : 저차원(2D) 평면에서 점 i 와 점 j 가 이웃할 확률 (거리와 값 비례, t-분포 기반으로 계산)

→ 손실함수를 0에 가깝게 만든다 → 고차원 이웃관계 F와 저차원 이웃관계 Q를 최대한 똑같이 만든다는 것

UMAP (Uniform Manifold Approximation and Projection)

지역 관계 보존 & 전역 구조까지 표현하는 위상수학 기반 기법
두 데이터 사이 구조의 차이를 측정하는 Cross-Entropy를 최소화하는것이 목표

$$L_{UMAP} = \sum_{i \neq j} \left(p_{ij} \log \left(\frac{p_{ij}}{q_{ij}} \right) + (1 - p_{ij}) \log \left(\frac{1 - p_{ij}}{1 - q_{ij}} \right) \right)$$

p_{ij} : 고차원 공간에서 점 i,j가 이웃일 확률. t-SNE와 유사하지만 위상수학 그래프 이론에 기반해 계산됨

q_{ij} : 저차원에서 점 i,j가 연결되어 있을 확률(가중치)

→ 밀어내고 당기며 t-SNE보다 안정적인 구조 만듦

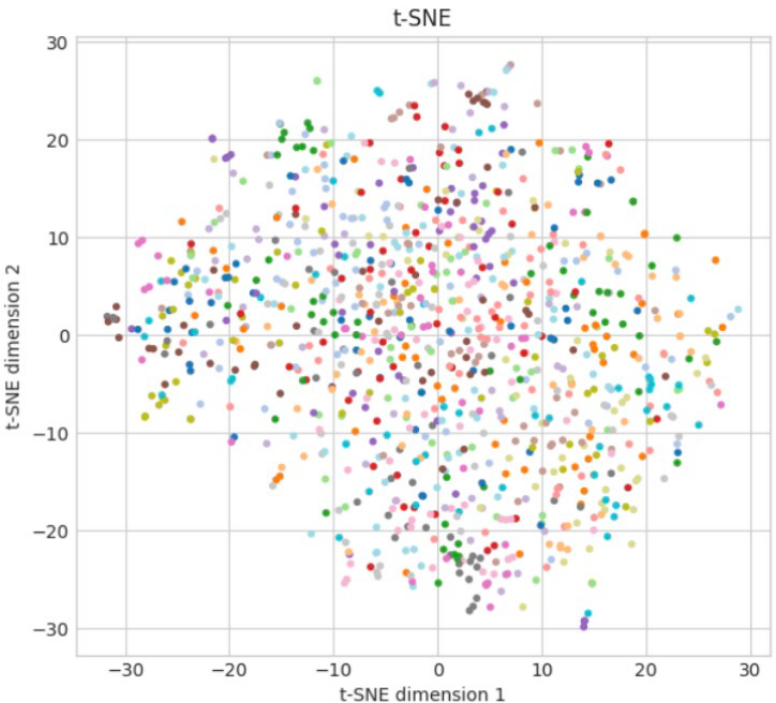
VAE (Variational Autoencoder)

시각화가 아닌 새로운 데이터를 생성하는것이 목표
Encoder의 압축, decoder의 복원
ELBO라는 목적식을 최대화 (음수 ELBO를 최소화)하는 것

$$L_{VAE} = \underbrace{\mathbb{E}[\log p(x|z)]}_{\text{Reconstruction Loss}} - \underbrace{D_{KL}(q(z|x)||p(z))}_{\text{Regularization Loss}}$$

1. Reconstruction Loss: 압축된 벡터 z로부터 원본 이미지 x를 얼마나 잘 복원했는지 측정 (손실이 작을수록 이미지를 잘 압축)
2. KL Divergence : 인코더가 만들어낸 압축 공간이 미리 정해놓은 표준 정규 분포와 비슷해지도록 강제 (압축 공간이 부드럽게 이어져 새로운 데이터를 생성하기 좋아짐)

t-SNE (t-Distributed Stochastic Neighbor Embedding)

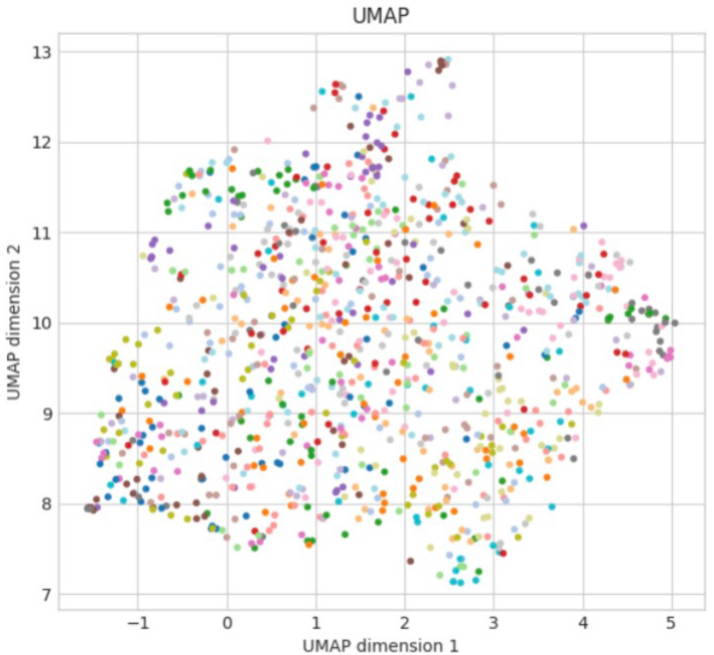


·이웃 관계에만 집중

→ 같은 클래스(이웃)끼리는 매우 촘촘하게 뭉쳐있지만, 군집과 군집 사이의 거리는 수학적으로 큰 의미를 갖지 않는 분리된 군집들이 나타남

t-분포를 사용 → 군집 간 분리가 더 극적

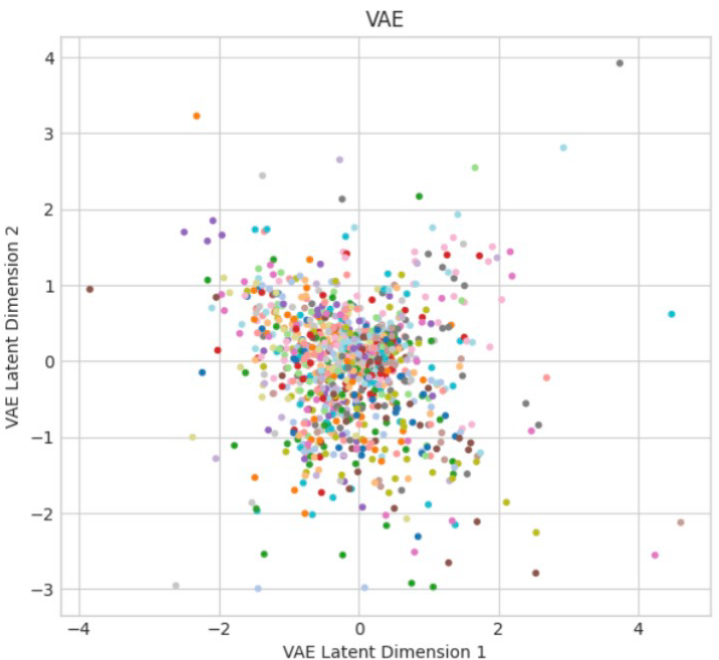
UMAP (Uniform Manifold Approximation and Projection)



가까운 점을 당기는 힘과 먼 점을 미는 힘의 균형

t-SNE처럼 지역적인 군집을 형성하면서도, 군집들 사이의 관계나 데이터의 전체적인 모양이 어느 정도 유지

VAE (Variational Autoencoder)



Loss function에 KL divergence가 포함,
재구성 손실보다 모든 점을 원점 근처의 원형 분포로 모으는 힘이 더 강하기 때문에 중앙에 몰려있음

→ 분류시각화에는 나쁨, 부드러운 생성 공간 만듦에는 좋음

VAE & GAN

두 모델 모두 새로운 데이터를 만들어내는 생성 모델

VAE : encoder – decoder
GAN : 생성자 – 판별자

VAE

(Auto-Encoding Variational Bayes, Diederik et al., 2013)

잘 압축하면 잘 생성할 수 있다
Data를 요약하는 encoder와, 요약본을 바탕으로 원본을 복원하는 decoder로 구성

1. 인코더 (초상화가 1): 어떤 사람의 얼굴(x)을 보고, 그 사람의 모든 특징(눈 모양, 코 높이, 피부 톤 등)을 작은 메모지(잠재 공간, z)에 핵심만 요약해서 적음.
이때 "눈: 3cm"처럼 정확한 값이 아닌, "눈 크기: 평균 3cm에 표준편차 0.2cm 정도" 와 같이 확률적 분포(대략적) 기록

2. 디코더 (초상화가 2): 첫 번째 화가가 넘겨준 메모지(z)만 보고 원래 사람의 얼굴을 똑같이 그려냄.

→ 메모지에 살짝 다른 값 적어넣으면 새로운 data 자연스럽게 생성

GAN

Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks(Alex et al., 2015)

생성자(generator)와 판별자(discriminator)의 경쟁

1. Generator, 위조지폐범: 아무렇게나 그린 그림(랜덤 노이즈)으로 위조지폐를 만들기 시작.
경찰의 반응을 보며 점점 더 진짜 같은 위조지폐를 만듦

2. Discriminator, 경찰: 진짜 지폐와 생성자가 만든 위조지폐를 구별
진짜와 위조를 더 잘 구별하려고 발전

불안정했던 초기 GAN에 CNN(합성곱 신경망구조를 안정적으로 적용하는 방법인(DCGAN)을 제시
→ CNN 설계 방법 (pooling 대신 stride, FC제거, 배치정규화 사용, 적절한 activation function)

Pooling for multi class

Cifar-100, ResNet-18 based, only different pooling methods

최종 실험 결과 요약

Pooling Method	Best Test Accuracy	Training Time (min)	Final GeM 'p'
Average Pooling	0.5468	107.54	-
Max Pooling	0.5530	109.60	-
GeM Pooling	0.5508	114.46	0.0764
Attention Pooling	0.5492	110.84	-

Pooling for multi label

Img 특성

CIAFR-100 : global features
NIH : localized features

분류 과제

CIAFR-100 : multi classes
NIH : multi labels

풀링 역할

CIAFR-100 : 차원축소, 본 이미지 불변성 확보
NIH : 병변 특징 추출, 증폭(가중치) → attention pooling

Feedback

예상 결과 증명해보기

Latent space(z) 더 공부하기

World Models,
Ha, Schmidhuber (2018)

At each time step, our agent receives an **observation** from the environment.

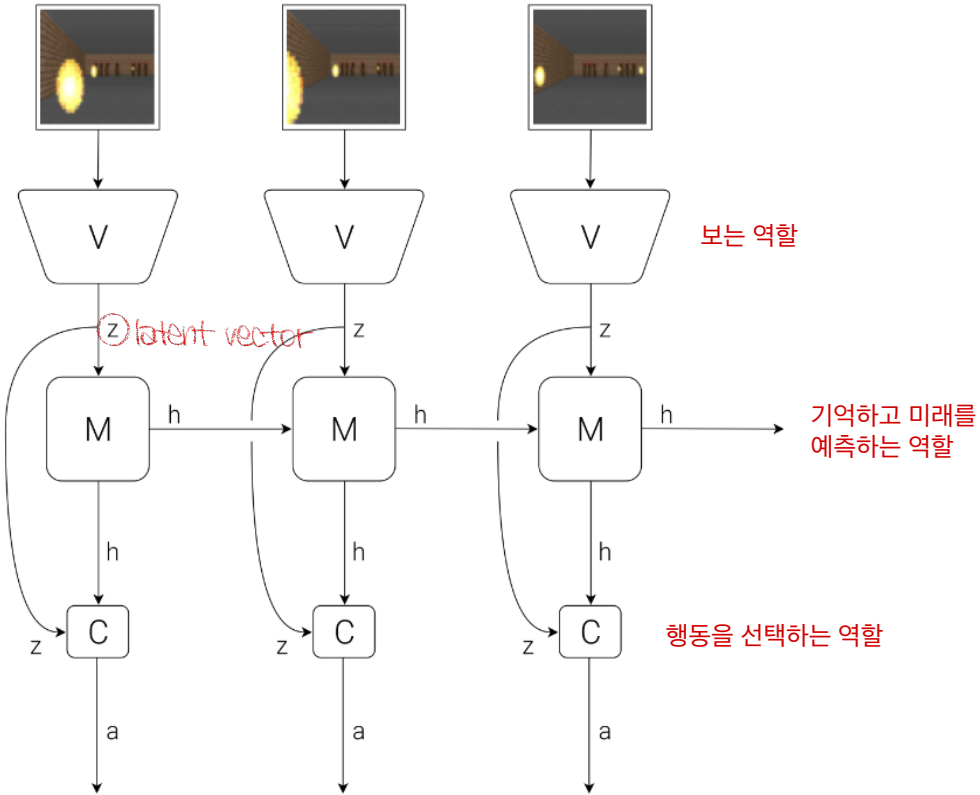
World Model

The **Vision Model (V)** encodes the high-dimensional observation into a low-dimensional latent vector.

The **Memory RNN (M)** integrates the historical codes to create a representation that can predict future states.

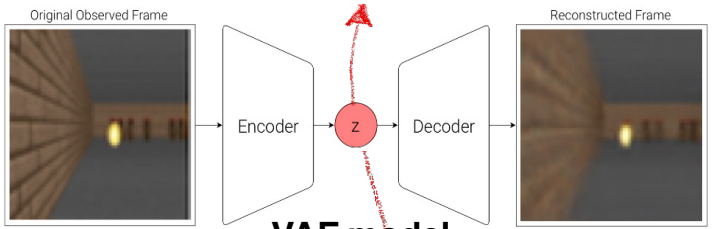
A small **Controller (C)** uses the representations from both **V** and **M** to select good actions.

The agent performs **actions** that go back and affect the environment.



World Models,
Ha, Schmidhuber (2018)

학습 데이터, 최대의 z



원본 이미지

VAE model
(V)

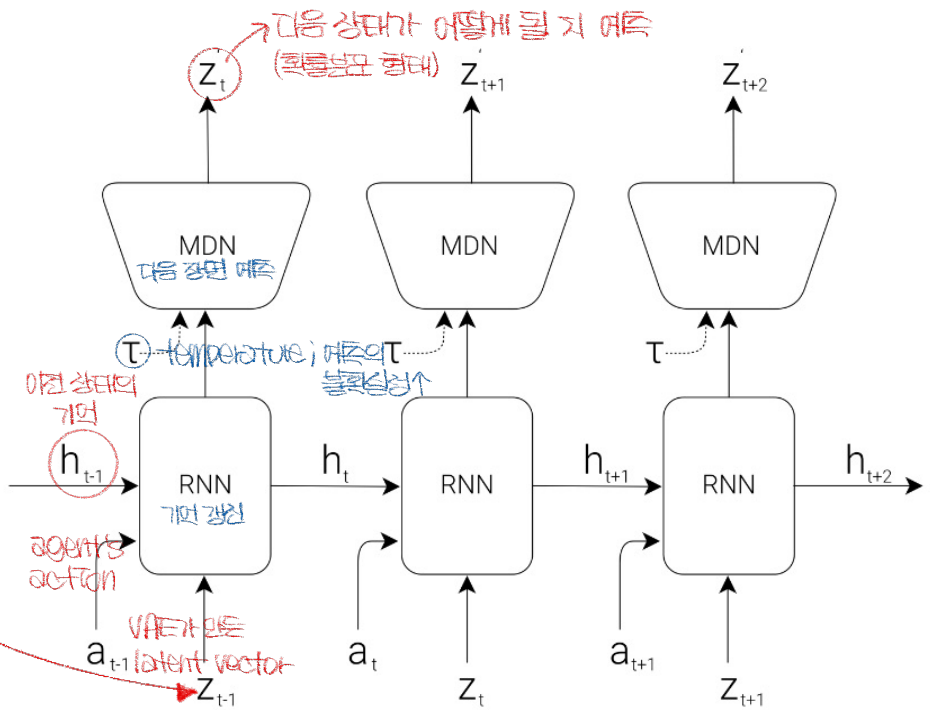
재구성된 이미지

*z : 잠재 벡터(latent vector)

이미지의 중요한 특징을 숫자로 표현
핵심 정보만 남김

V의 latent vector – 실제 관찰에서 얻음
의 latent vector – 기존의 정보로 예측

→ 모두 같은 공간(차원 구조, 의미체계)
같은 latent space 안에서 공유



RNN : 시간에 따라 들어오는 정보를 순서대로 기억하고 처리하는 신경망

$z_{t+1} \sim P(z \mid z_t, a_t, h_t)$

MDN-RNN model
(M)

World Models,
Ha, Schmidhuber (2018)

Controller가 Vision + Memory 정보를 바탕으로 행동을 선택

$$a_t = W_c \left[z_t \; h_t \right] + b_c$$

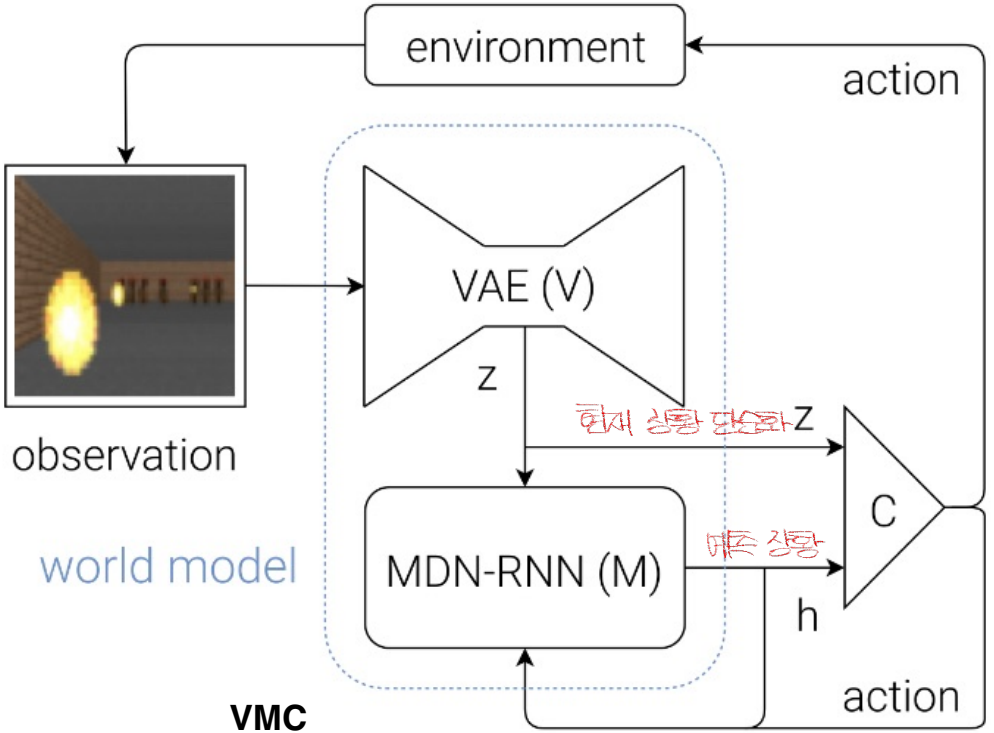
Controller model

(C)

Wc, bc : 선형변환을 위한 weight, bias

zt: vision model의 latent vector

ht: Memory RNN의 '과거 정보 + 미래 예측' 벡터



VMC

VAE + MDN-RNN + Controller

KL divergence,
Cross-Entropy & Augmentation